

Assignment 1

For client client server architecture, the choices for data serialization are numerous, for this kind of project choosing YAML for config is good choice as it human readable, and even though even though the YAML config is slow to parse, the config only loads once during application startup and the impact on performance is very low. For the protocol for server and client communication, protobuf is favorable because of it sends the data in binary format making the overall communication use very less bandwidth and faster as it does not need conversions from text to binary since it is already in binary, the server and the client also need strict contract so gRPC with protobuf schema enforces this. For RestAPIs that are exposed to consumers from the server either using live websockets or maybe GraphQL, JSON/XML formats are a standard since they will need to actually read the content and use it as they need and also we don't expect every consumer client to download our protobuf schema and start to implement their code accordingly, so a human readable format is required for the to know exactly what data they'll be working with on return of the API call without further deserialization.

When requesting historical-data API, REST fits well because of its flexibility in making a range, like you can use placeholders in params of the REST url and update when you want dynamically, also REST caches data through CDNs, for instance if 1000 calls request data from last week up to today morning, it can cache the response and the next upcoming request won't bother the server and database hence increasing speed and freeing up the server to serve other requests. In contrast, SOAP is function driven and relies on complex XML envelopes. While it offers strong built-in security, it is heavyweight, and less suitable for public APIs consumed by lightweight clients such as mobile apps. Traditional RPC focuses on calling remote procedures rather than manipulating resources.

This project is I/O-bound and coroutines and event loops are preferred over threads because the server is only receiving data from the farms and storing the readings in the database, no processing is being performed that is heavy and needs to be processed in parallel hence no introduction of threads. For the clients requesting the live data too, they do not need threads as they will just call a live websocket or a rest API call to fetch historic data and no parallel execution needed. It would be a better practice to use threads when the server would need to start process the data it is getting from the farms and then the sensors at the same time would want just to post and receive a feed back that the data has been posted, in this way we would receive the data and check if its valid and introduce a thread to work on processing at the same time returning the function so the sensor would stop blocking and continue monitoring while the server go on processing the received data.

The server and the consuming clients requesting the live data should have a standard and negotiate the type of content that would be sent and received, since the server would be taking that data in the database they should be a content negotiation that is, is the data being sent in JSON or XML, what are the fields being given on the

exposed API endpoints. This affects the design as we have to keep in mind when exposing the API, the return DTO should contain these fields and the client asking for the live websocket should expect this kind of data. The server and sensors also have to negotiate by sharing the schema of the protobuf. There should also be a persistent connection between the sensors and the server and also the server with the websocket. The server would need constant updates on the farms and keep those values in the database, while the users connected through the websocket would need live data meaning the server should constantly send data to the client meaning the connection should be persisted on both the sensors and the server to the clients.